

PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

SERVICIO NACIONAL DE APRENDIZAJE

Regional por diligenciar · Centro de formación por diligenciar

GUÍA 05

Verificación y aseguramiento de la calidad del software mediante pruebas automatizadas (Testing & QA)

Programa Análisis y Desarrollo de Software - ADSO

Fase del proyecto: Evaluación — Calidad de Software

Adaptación didáctica · Versión 3.0.0 · 10 de septiembre de 2026

1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

Denominación del Programa de Formación: Análisis y Desarrollo de Software

Código del Programa de Formación: 228118

Nombre del Proyecto Formativo: Construcción de software a la medida para gestionar inventario y calidad del sistema de información del proyecto formativo.

Fase del Proyecto: Evaluación

Actividad de Proyecto Formativo: Verificar la calidad y funcionalidad de los entregables del sistema de información mediante pruebas automatizadas.

Competencia: Verificar los entregables del desarrollo de software de acuerdo con las especificaciones del diseño (código 220501098).

Resultados de Aprendizaje:

1. Diseñar y ejecutar el plan de pruebas de software verificando el cumplimiento de los requisitos funcionales y no funcionales.
2. Implementar y automatizar las pruebas unitarias, de integración y end-to-end del sistema de información según el plan de pruebas y los umbrales de cobertura establecidos (articulación).

Duración de la Guía de Aprendizaje: 40 horas (10 de teoría y 30 de práctica)

Modalidad: Presencial apoyada en plataforma virtual

Versión de la adaptación: 3.0.0

2. PRESENTACIÓN

El desarrollo de software no termina cuando el código compila: termina cuando el sistema demuestra que hace lo que debe hacer y resiste las condiciones reales de uso. Esta guía cierra el ciclo del proyecto formativo entregando al aprendiz la caja de herramientas para VERIFICAR: pruebas unitarias, de integración y end-to-end, gestión de la cobertura, automatización en CI/CD y auditoría de la calidad.

La calidad no es una fase final ni una responsabilidad de un especialista: es una práctica de equipo. Un defecto detectado en diseño cuesta una fracción de lo que cuesta en producción; los casos reales demuestran que una sola omisión puede hacer que un sistema funcione bien en el laboratorio y colapse en la operación.

El caso transversal es un sistema de información de inventario y calidad. Sobre ese entorno se trabajan los artefactos de las plataformas PyTest (FastAPI y Flask), Vitest/Jest (React), JUnit 5 & Mockito (Java) y Playwright (E2E). Si el aprendiz continúa un proyecto previo, debe reemplazar los datos del ejemplo por los de su propio repositorio.

Pregunta orientadora

¿Cómo garantizar que el software construido entrega el valor esperado y que un defecto no detectado no lo derribe en producción? El caso de Knight Capital (2012) perdió 440 millones de dólares en 45 minutos por código de prueba sin eliminar.

2.1 Producto integrador

Registro integral de evidencias del plan de pruebas: plan de pruebas IEEE 829, suite automatizada con cobertura superior o igual al 80 % en la lógica de negocio, pipeline de CI/CD funcional y bitácora V.E.R.A. (Verificar en documentos, Ejecutar con TDD, Revisar calidad y seguridad, Atribuir en bitácora).

2.2 Conocimientos previos

- Fundamentos de programación en al menos un lenguaje del ecosistema (Python, JavaScript o Java).
- Conceptos de HTTP, APIs REST y bases de datos.
- Uso básico de Git y repositorios remotos.
- Estructura de proyectos web y de microservicios.

2.3 Ruta de aprendizaje

- Reflexionar sobre el costo de los defectos no detectados.
- Comprender la pirámide de pruebas y el flujo lógico de 7 fases.
- Escribir pruebas unitarias con TDD y validar cobertura.
- Probar APIs, componentes y flujos completos (E2E).
- Integrar la calidad en el pipeline CI/CD y en Git.
- Auditar la calidad y la seguridad antes de desplegar.

Uso responsable de las herramientas de apoyo (incluida la IA)

Las herramientas de IA pueden ayudar a consultar la documentación compartida, proponer listas de chequeo, detectar vacíos entre actividad-evidencia-criterio y generar pruebas de referencia. No sustituyen la ejecución real de la suite ni la verificación humana de los resultados. Los códigos de programa, competencia, resultados de aprendizaje y duración deben validarse con el instructor antes de una entrega institucional. Esta guía incorpora además el sistema qa_auditor, que detecta anomalías del código generado con IA: pruebas vacías, imports alucinados y vulnerabilidades OWASP.

3. PROYECTO FORMATIVO Y ENSEÑANZA POR PROYECTOS

La guía se desarrolla con la estrategia de aprendizaje por proyectos: todo ejercicio de prueba nace de un entregable real del proyecto formativo y termina como evidencia verificable. El aprendiz trabaja sobre el sistema de información del caso transversal y debe sostener sus afirmaciones con ejecuciones reales (capturas de comando y reportes generados por las herramientas).

Fase del proyecto	Etapas del proyecto	Entregable	Criterio de éxito
Evaluación	Plan de pruebas	Plan de pruebas IEEE 829 (ART-TEST-01)	Define alcance, estrategia y matriz de casos de prueba.
Evaluación	Automatización	Suite de pruebas automatizadas (ART-TEST-02)	Pruebas unitarias, de integración y E2E con cobertura $\geq 80\%$.
Evaluación	Reporte y trazabilidad	Automatización E2E y registro de defectos (ART-TEST-03)	Ejecuta flujos Playwright y clasifica defectos con evidencia.

4. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

Las actividades siguen la secuencia SENA de reflexión, contextualización, apropiación y transferencia. El aprendiz debe conservar versiones, fuentes y retroalimentación en su registro de evidencias.

Actividad	Fase de la secuencia	Duración	Evidencia esperada
4.1 El bug de 440 millones	Reflexión inicial	4 horas	Respuesta argumentada al caso
4.2 Pirámide de pruebas y flujo de 7 fases	Contextualización	6 horas	Cuestionario y tabla de fases
4.3 PyTest, Vitest, JUnit 5 y Playwright con TDD/BDD	Apropiación	20 horas	Suite automatizada y cobertura $\geq 80\%$
4.4 Pipeline QA y auditoría del proyecto	Transferencia	10 horas	Registro integral de evidencias

4.1 Reflexión inicial: el bug de 440 millones de dólares

Propósito

Reconocer el costo humano, económico y reputacional de publicar software sin verificar su comportamiento, y valorar la prueba automatizada como una inversión y no como un costo.

Reto

Knight Capital, el 1 de agosto de 2012, ejecutó un algoritmo de trading con código de prueba no eliminado (SMBAT): tomó posiciones erróneas cada pocos segundos durante 45 minutos y perdió 440 millones de dólares. Un código de prueba de la propia campaña convirtió un sistema rentable en una ruleta. ¿Qué falló en las verificaciones previas al despliegue?

Pensamiento inicial

1. ¿Qué tipo de defecto pudo detectar una prueba automatizada y por qué no existía?
2. ¿Quién tomó la decisión de desplegar y con qué evidencia?
3. ¿Qué se habría automatizado para detener el despliegue en cinco segundos?

Cierre de la reflexión

Cada equipo redacta una conclusión de máximo 120 palabras. Debe explicar, con evidencia del caso, por qué la calidad verificada cuesta menos que la calidad asumida. Los comandos que se muestran a continuación son un punto de partida del laboratorio.

```
"""Configuración compartida de tests pytest para el microservicio FastAPI."""
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool
from main import app
from database import Base, get_db
```

```

SQLALCHEMY_DATABASE_URL = "sqlite://"
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

@pytest.fixture(scope="function")
def db_session():
    Base.metadata.create_all(bind=engine)
    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()
        Base.metadata.drop_all(bind=engine)

@pytest.fixture(scope="function")
def client(db_session):
    def override_get_db():
        try:
            yield db_session
        finally:
            db_session.close()
    app.dependency_overrides[get_db] = override_get_db
    yield TestClient(app)
    app.dependency_overrides.clear()

```

Componente	Orientación
Ambiente requerido	Aula con computador y proyector, o ambiente virtual colaborativo.
Estrategia o técnica didáctica activa	Aprendizaje basado en problemas y conversación guiada.
Materiales de formación	Caso impreso o digital y notas adhesivas.
Material de apoyo	Guía web 5, módulo de reflexión del bug de 440 millones.
Evidencia de aprendizaje	Respuesta argumentada al caso y matriz de pruebas del ecosistema.
Instrumento de evaluación	Lista de chequeo de participación y argumentación.
Duración	4 horas

4.2 Contextualización: pirámide de pruebas y Flujo Lógico Maestro de 7 Fases

Propósito

Construir el vocabulario y el modelo mental para decidir qué probar, con qué herramienta y en qué momento, según la pirámide de pruebas (70 % unitarias, 20 % integración, 10 % end-to-end).

Reto

Explicar con un ejemplo real por qué una prueba end-to-end no reemplaza una prueba unitaria, y qué costos asume un equipo que invierte la proporción.

Pirámide de pruebas

Para avanzar rápidamente en cualquier proyecto de software, el aprendiz debe abordar las pruebas de la base a la cúspide: 70 % unitarias, 20 % de integración y 10 % end-to-end.

Fase	Nivel	Herramientas	Objetivo técnico
1. Arnés	Configuración	venv, npm, pom.xml	Aislar dependencias y armar el directorio tests/
2. Dominio (70 %)	Unitarias	PyTest, Vitest, JUnit 5	Lógica pura, cálculos y modelos en menos de 5 ms
3. Integración (20 %)	API y BD	TestClient, MockMvc	Endpoints HTTP y base de datos de prueba limpia
4. Componentes	Frontend UI	React Testing Library	DOM virtual y accesibilidad con getByRole
5. E2E (10 %)	Cúspide	Playwright	Caminos dorados en navegador real headless
6. Cobertura	Calidad	pytest-cov, JaCoCo, qa_auditor	Validar cobertura $\geq 80\%$ y OWASP SAST
7. CI/CD	Automatización	GitHub Actions	Compuerta de calidad en cada push antes de tocar el VPS

Procedimiento

1. Clasificar diez ejemplos de prueba como unitaria, de integración o end-to-end.
2. Justificar los porcentajes de la pirámide en términos de velocidad y confiabilidad.
3. Asociar cada una de las 7 fases con su herramienta y su objetivo.
4. Completar el simulador de la pirámide de la guía web.

Componente	Orientación
Ambiente requerido	Aula de informática con navegador y guía web interactiva.
Estrategia o técnica didáctica activa	Exposición dialogada y aprendizaje colaborativo.
Materiales de formación	Tabla de fases, fichas de conceptos y ejemplos del caso transversal.
Material de apoyo	Guía web 5, sección «Pirámide de pruebas» y simulador.
Evidencia de aprendizaje	Cuestionario de conceptos y tabla de fases resuelta.
Instrumento de evaluación	Cuestionario y lista de chequeo del modelo.
Duración	6 horas

4.3 Apropiación: automatización de pruebas con TDD y BDD

Propósito

Automatizar las pruebas del caso transversal con las cuatro plataformas del ecosistema (PyTest, Vitest/Jest, JUnit 5 y Playwright), aplicando TDD y BDD, hasta alcanzar una cobertura mínima del 80 % en la lógica de negocio.

Reto

Redactar primero las pruebas y después la implementación (rojo, verde, refactor): la suite debe poder ejecutarse en un entorno limpio con un solo comando y fallar con diagnóstico claro.

4.3.1 PyTest: microservicio FastAPI

Automatización de pruebas de endpoints REST con TestClient y base de datos aislada en memoria.

Paso a paso para pruebas automatizadas en FastAPI:

Paso 1: Entorno. Instalar PyTest, el cliente de prueba HTTP y la herramienta de cobertura.

```
> pip install pytest pytest-cov httpx fastapi sqlalchemy pydantic
```

Regla de oro: Usa httpx para que TestClient no requiera levantar un servidor de red real.

Paso 2: Aislamiento (conftest.py). Crear fixtures con SQLite en memoria y dependency_overrides para inyectar una sesión limpia por cada prueba.

```
> pytest tests/ -v
```

Regla de oro: Llama app.dependency_overrides.clear() al terminar cada prueba.

Paso 3: Redacción AAA. Escribir los casos de prueba con el patrón Arrange, Act, Assert validando códigos de respuesta y JSON.

```
> pytest tests/test_productos.py -v
```

Regla de oro: Valida códigos de error como 404 y 422 además del caso exitoso 201.

Paso 4: Ejecución en CLI. Ejecutar la suite con banderas de diagnóstico rápido.

```
> pytest tests/ -v -s -x --tb=short
```

Regla de oro: La bandera -x detiene la ejecución en el primer fallo para resolverlo de inmediato.

Paso 5: Cobertura. Verificar que la lógica alcance al menos el 80 % de cobertura.

```
> pytest --cov=app --cov-report=term-missing
```

Regla de oro: Inspecciona response.json()['detail'] si un endpoint responde 422.

```
"""Tests del CRUD de productos del microservicio FastAPI."""
from fastapi import status

def test_crear_producto(client):
    response = client.post("/productos/", json={
        "nombre": "Laptop HP",
        "precio": 1500.00,
```

```

        "stock": 10,
        "categoria_id": 1,
    })
    assert response.status_code == status.HTTP_201_CREATED
    data = response.json()
    assert data["nombre"] == "Laptop HP"
    assert "id" in data

def test_listar_productos_vacio(client):
    response = client.get("/productos/")
    assert response.status_code == 200
    assert response.json() == []

def test_obtener_producto_no_existente(client):
    response = client.get("/productos/9999")
    assert response.status_code == status.HTTP_404_NOT_FOUND

def test_actualizar_producto(client):
    r = client.post("/productos/", json={"nombre": "Mouse", "precio": 25.0,
    "stock": 50})
    pid = r.json()["id"]
    response = client.put(f"/productos/{pid}", json={"nombre": "Mouse Logitech",
    "precio": 35.0})
    assert response.status_code == 200
    assert response.json()["nombre"] == "Mouse Logitech"

def test_soft_delete(client):
    r = client.post("/productos/", json={"nombre": "Teclado", "precio": 40.0,
    "stock": 5})
    pid = r.json()["id"]
    response = client.delete(f"/productos/{pid}")
    assert response.status_code == 204

def test_validacion_precio_negativo(client):
    response = client.post("/productos/", json={"nombre": "X", "precio": -10,
    "stock": 5})
    assert response.status_code == status.HTTP_422_UNPROCESSABLE_CONTENT

def test_validacion_precio_cero(client):
    response = client.post("/productos/", json={"nombre": "X", "precio": 0,
    "stock": 5})
    assert response.status_code == status.HTTP_422_UNPROCESSABLE_CONTENT

```

4.3.2 PyTest: aplicación web Flask

Automatización de las rutas, plantillas, formularios y sesiones de la aplicación web Flask que construiste en la guía anterior (Guía Flask). Todos los pasos se ejecutan en la carpeta raíz de ese proyecto: la que contiene run.py, requirements.txt, la carpeta app/ y la carpeta tests/ con test_routes.py. Si no tienes el proyecto en tu computador, clónalo desde el repositorio que indique tu instructor o pídelo como entrega de la fase anterior antes de continuar.

Paso a paso para preparar el proyecto y ejecutar las pruebas:

Paso 1: Abrir el terminal en la raíz del proyecto. Abre PowerShell (menú inicio y escribe PowerShell) y navega hasta la carpeta del proyecto Flask. Reemplaza la ruta del ejemplo por la tuya.

```
> cd "C:\Users\TuUsuario\Guia-Flask"
```

Regla de oro: En PowerShell puedes arrastrar la carpeta sobre la ventana para pegar su ruta completa.

Paso 2: Verificar el contenido del proyecto. Lista los archivos de la carpeta para confirmar que estás en el lugar correcto.

```
> Get-ChildItem
```

Regla de oro: Deben aparecer run.py, requirements.txt, la carpeta app/ y la carpeta tests/.

Paso 3: Crear el entorno virtual. Crea la carpeta venv que aísla Python y las librerías del proyecto para no mezclarlas con otros proyectos del sistema.

```
> python -m venv venv
```

Regla de oro: Si PowerShell responde «python no se reconoce», instala Python desde python.org marcando la casilla Add Python to PATH y abre un terminal nuevo.

Paso 4: Activar el entorno virtual. Activa el entorno para que pip y pytest usen las librerías del proyecto.

```
> .\venv\Scripts\activate
```

Regla de oro: El prompt debe cambiar y mostrar (venv) al inicio. Si PowerShell bloquea el script por políticas de ejecución, ejecuta antes Set-ExecutionPolicy -Scope Process Bypass.

Paso 5: Instalar las dependencias de la aplicación. Instala Flask y sus extensiones leyendo la lista del archivo requirements.txt del proyecto.

```
> pip install -r requirements.txt
```

Regla de oro: El comando debe ejecutarse en la raíz del proyecto; si aparece el error de archivo no encontrado, revisa la carpeta en la que estás.

Paso 6: Instalar las herramientas de prueba. Instala el ejecutor de pruebas y el medidor de cobertura.

```
> pip install pytest pytest-cov
```

Regla de oro: pytest ejecuta las pruebas y pytest-cov mide el porcentaje de código cubierto.

Paso 7: Levantar la aplicación para verificar (opcional). Arranca el servidor de desarrollo y abre http://127.0.0.1:5000 en el navegador. Para detenerlo vuelve al terminal y presiona Ctrl+C.

```
> python run.py
```

Regla de oro: No es obligatorio para las pruebas (el test_client no usa red), pero confirma que el entorno quedó bien configurado.

Paso 8: Ejecutar la suite completa. Ejecuta todas las pruebas del proyecto. PyTest lee pytest.ini (testpaths = tests) y reporta cada prueba en pantalla.

```
> pytest
```

Regla de oro: La última línea resume el resultado: N passed si todo pasó o M failed si algo falló; corrige los fallos de arriba hacia abajo.

Paso 9: Ejecutar el archivo de rutas. Corre solo las pruebas del archivo test_routes.py (rutas, plantillas y sesiones).

```
> pytest tests/test_routes.py -v
```

Regla de oro: La bandera -v (verbose) muestra el nombre y el estado de cada prueba individual.

Paso 10: Ejecutar una prueba puntual. Corre un único caso de prueba usando el formato archivo::prueba.

```
> pytest tests/test_routes.py::test_health_check -v
```

Regla de oro: El separador :: indica a PyTest exactamente qué prueba ejecutar dentro del archivo.

Paso 11: Medir la cobertura. Verifica el porcentaje de líneas ejecutadas por las pruebas en el paquete app y exige el umbral mínimo del 80 %.

```
> pytest --cov=app --cov-report=term-missing --cov-fail-under=80
```

Regla de oro: --cov-report=term-missing agrega la columna Missing con las líneas que ninguna prueba ejecutó; si la cobertura baja del 80 %, la ejecución termina en error.

Si un comando responde ModuleNotFoundError, revisa en este orden: ¿estás en la raíz del proyecto?, ¿el entorno está activo (prompt con (venv))?, ¿instalaste requirements.txt y las herramientas de prueba?

4.3.3 Vitest: componentes React

Paso a paso para pruebas automatizadas en React:

Paso 1: Dependencias. Instalar Vitest, React Testing Library y jsdom.

```
> npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

Regla de oro: jsdom emula el DOM del navegador en Node.js de forma rápida.

Paso 2: Configuración. Configurar environment: 'jsdom' y setupFiles en vitest.config.js.

```
> npm test
```

Regla de oro: jest-dom agrega aserciones como .toBeInTheDocument() y .toBeDisabled().

Paso 3: Accesibilidad. Renderizar el componente y buscar elementos mediante screen.getByRole.

```
> npm test -- Contador.test.jsx
```

Regla de oro: Busca botones por su texto visible, no por selectores de clase CSS.

Paso 4: Eventos. Simular clics con fireEvent o user-event y verificar cambios de estado.

```
> npm run test:watch
```

Regla de oro: Ante actualizaciones asíncronas, usa `waitFor(() => expect(...))`.

Paso 5: Cobertura del frontend. Generar reporte de cobertura V8 para componentes.

```
> npm run test:coverage
```

Regla de oro: Asegúrate de probar los estados de cargando, error y éxito.

```
import React from "react";
import { render, screen, fireEvent } from "@testing-library/react";
import { describe, it, expect } from "vitest";
import Contador from "../src/components/Contador.jsx";

describe("Contador", () => {
  it("inicia en 0", () => {
    render(<Contador />);
    expect(screen.getByText(/contador: 0/i)).toBeInTheDocument();
  });

  it("incrementa al hacer clic", () => {
    render(<Contador />);
    fireEvent.click(screen.getByRole("button", { name: /incrementar/i }));
    expect(screen.getByText(/contador: 1/i)).toBeInTheDocument();
  });

  it("decrementa al hacer clic", () => {
    render(<Contador />);
    fireEvent.click(screen.getByRole("button", { name: /decrementar/i }));
    expect(screen.getByText(/contador: -1/i)).toBeInTheDocument();
  });
});
```

4.3.4 JUnit 5 & Mockito: backend Java

Paso a paso para pruebas automatizadas en Java:

Paso 1: pom.xml de Maven. Agregar dependencias junit-jupiter-engine y mockito-core con alcance de prueba.

```
> mvn dependency:resolve
```

Regla de oro: Asegura la compatibilidad con `maven-surefire-plugin 3.x`.

Paso 2: Estructura de la clase. Crear la clase de prueba en `src/test/java/` con `@ExtendWith(MockitoExtension.class)`.

```
> mvn test-compile
```

Regla de oro: Usa `@DisplayName` para documentar la intención en español.

Paso 3: Mocks y partes simuladas. Crear dobles con `@Mock` y definir respuestas simuladas con `when().thenReturn()`.

```
> mvn test -Dtest=ProductoDAOTest
```

Regla de oro: Simula `SQLException` para verificar la recuperación ante fallos.

Paso 4: Aserciones y verificación. Validar resultados con assertEquals y verificar llamadas con verify(mockStmt).executeUpdate().

```
> mvn test
```

Regla de oro: verify(mock, never()) garantiza que los métodos peligrosos no fueron invocados.

Paso 5: Reporte JaCoCo. Generar reporte de cobertura con JaCoCo.

```
> mvn test jacoco:report
```

Regla de oro: Revisa target/site/jacoco/index.html en el navegador.

```
import org.junit.jupiter.api.*;
import org.mockito.Mockito;
import java.sql.*;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@DisplayName("ProductoDAO - Tests del CRUD")
class ProductoDAOTest {

    private Connection mockConn;
    private PreparedStatement mockStmt;
    private ResultSet mockRs;

    @BeforeEach
    void setUp() throws SQLException {
        mockConn = mock(Connection.class);
        mockStmt = mock(PreparedStatement.class);
        mockRs = mock(ResultSet.class);
        when(mockConn.prepareStatement(anyString())).thenReturn(mockStmt);
    }

    @Test
    @DisplayName("crear producto ejecuta INSERT correctamente")
    void testCrearProducto() throws SQLException {
        when(mockStmt.executeUpdate()).thenReturn(1);
        Producto p = new Producto(0, "Laptop", 1500.0, 10, "Electronica");
        ProductoDAO dao = new ProductoDAO(mockConn);
        boolean result = dao.crear(p);
        assertTrue(result);
        verify(mockStmt).executeUpdate();
    }

    @Test
    @DisplayName("listar productos retorna lista")
    void testListarProductos() throws SQLException {
        when(mockStmt.executeQuery()).thenReturn(mockRs);
        when(mockRs.next()).thenReturn(true, false);
        when(mockRs.getInt("id")).thenReturn(1);
        when(mockRs.getString("nombre")).thenReturn("Mouse");
        when(mockRs.getDouble("precio")).thenReturn(25.0);
        ProductoDAO dao = new ProductoDAO(mockConn);
    }
}
```

```

        List<Producto> productos = dao.listar();
        assertEquals(1, productos.size());
    }
}

```

4.3.5 Metodología TDD: ciclo rojo, verde, refactor

El ciclo fundamental de TDD consiste en 3 pasos iterativos cortos (menos de 5 minutos):

1. ROJO: escribir una prueba unitaria pequeña que falle antes de escribir el código de producción.
2. VERDE: escribir la implementación mínima para hacer pasar la prueba.
3. REFACTOR: limpiar el código y eliminar la duplicación manteniendo la prueba en verde.

4.3.6 BDD con Behave y Gherkin

La especificación por comportamiento conecta al equipo de negocio con el equipo técnico: cada escenario se lee como una historia que se escribe con la estructura Dado, Cuando, Entonces.

Paso a paso para BDD:

Paso 1: Instalación. Instalar behave y crear las carpetas features/ y features/steps/.

```
> pip install behave requests
```

Regla de oro: Gherkin conecta al equipo de negocio con el equipo técnico.

Paso 2: Escenario. Redactar el archivo .feature con estructura Given, When, Then.

```
> behave features/
```

Regla de oro: Usa Scenario Outline para probar tablas de combinaciones.

Paso 3: Pasos en Python. Implementar las funciones con @given, @when, @then pasando datos por context.

```
> behave
```

Regla de oro: context.response almacena las respuestas entre los pasos.

4.3.7 Playwright con Flask y Jinja

Como ya dominas Flask, no necesitas aprender otro frontend para empezar. La pregunta es qué ocurre con tu aplicación cuando una persona la usa: una ruta Flask llama a render_template(), Jinja produce HTML, el navegador muestra ese HTML y una acción del usuario genera otra petición. Playwright automatiza y comprueba esa cadena completa.

Playwright no reemplaza a PyTest ni al test_client de Flask. PyTest prueba la vista y las reglas del servidor sin abrir un navegador; Playwright comprueba el camino crítico desde la interfaz, con un navegador real.

Mapa mental de Flask/Jinja a Playwright

Flask/Jinja	Playwright	Qué se comprueba
@app.get('/productos')	page.goto('/productos')	La ruta carga.
render_template()	getByRole() / getByTestId()	El HTML aparece y es

		localizable.
label + input	getByLabel().fill()	El campo es usable como lo percibe el usuario.
request.form	click()	El formulario se envía.
redirect(url_for())	expect(page).toHaveURL()	La navegación termina donde debe.
flash() en una plantilla	getByRole('status','alert')	El resultado se comunica.

Herramientas esenciales

Herramienta	Modelo mental	Uso en Flask/Jinja
test y fixtures	Organizan el caso y su preparación.	Crear contexto limpio para cada flujo.
browser / context	Motor y sesión aislada.	Separar usuarios, cookies y almacenamiento.
page	Una pestaña del navegador.	Navegar, observar y ejecutar acciones.
locator	Referencia viva a un elemento.	Encontrar campos, botones, filas y mensajes.
expect	Aserción que espera.	Comprobar URL, texto, visibilidad y estado.
request	Cliente HTTP auxiliar.	Preparar datos sin recorrer la interfaz.
route	Control de peticiones.	Aislar una dependencia externa cuando aplique.
trace, report y codegen	Evidencia y diagnóstico.	Revisar DOM, red, consola y acciones grabadas.

Paso a paso para probar la aplicación Flask

Paso 1: Abrir el ejemplo correcto. Trabaja desde recursos/codigo-ejemplo/. El ejemplo flask_jinja_demo/ tiene una ruta de productos, un formulario POST, validación, redirect, flash y plantillas Jinja.

Regla de oro: Primero identifica el comportamiento que quieres probar; después eliges el localizador y la aserción.

Paso 2: Instalar Playwright y Chromium. Instala el ejecutor de pruebas y el navegador que usará la suite. La clave local se define en el entorno porque Flask necesita firmar los mensajes flash.

```
> npm install -D @playwright/test; npx playwright install chromium;
$env:FLASK_SECRET_KEY = "<CLAVE_LOCAL_DE_PRUEBA>"
```

Regla de oro: Nunca copies credenciales de producción a los tests ni guardes la clave local en Git.

Paso 3: Conectar el ejecutor con Flask. playwright.flask.config.js declara baseURL y webServer. Playwright inicia flask run en el puerto 5017 antes de correr los casos y reutiliza un servidor local si ya está levantado.

```
> npx playwright test --config=playwright.flask.config.js
```

Regla de oro: Con baseURL usa rutas relativas: page.goto('/productos/'), no URLs repetidas en cada prueba.

Paso 4: Localizar como lo haría el usuario. Prefiere getByRole y getByLabel; usa getByTestId cuando necesitas un contrato explícito. La etiqueta Jinja label for=nombre permite localizar el input sin acoplarse a clases CSS.

Regla de oro: Evita XPath y selectores como `div > table > tr:nth-child(2)`: describen la maquetación, no la intención.

Paso 5: Afirmar el resultado completo. Comprueba la URL después del redirect, el mensaje flash y la fila que Jinja agregó a la lista. En el caso inválido comprueba el alert y que no apareció un producto nuevo.

```
> npx playwright test --config=playwright.flask.config.js --ui
```

Regla de oro: Una acción sin una aserción observable no demuestra que el requisito se cumplió.

Paso 6: Diagnosticar sin esperas fijas. Usa `headed` para ver el navegador, UI para avanzar paso a paso, `codegen` para descubrir una interacción y el reporte con `trace` para inspeccionar DOM, red, consola y capturas.

```
> npx playwright test --config=playwright.flask.config.js --headed; npx playwright show-report
```

Regla de oro: Confía en las esperas automáticas de `expect` y localizadores; `setTimeout` solo oculta problemas de sincronización.

Código del ejemplo

Lee los archivos en este orden: la vista Flask define el comportamiento; la plantilla Jinja define el HTML que el navegador recibe; la configuración conecta el servidor; el spec describe el flujo y sus resultados.

Vista Flask: `flask_jinja_demo/app.py`

```
"""Aplicacion Flask minima para practicar Playwright sobre HTML renderizado."""

import os

from flask import Flask, flash, redirect, render_template, request, session,
url_for

def create_app():
    app = Flask(__name__)
    secret_key = os.environ.get("FLASK_SECRET_KEY")
    if not secret_key:
        raise RuntimeError("Define FLASK_SECRET_KEY antes de iniciar la
demostracion.")
    app.config["SECRET_KEY"] = secret_key
    @app.get("/")
    def home():
        return redirect(url_for("products_view"))

    @app.route("/productos", methods=["GET", "POST"])
    def products_view():
        products = session.get("products", [])
        if request.method == "POST":
            name = request.form.get("nombre", "").strip()
            price_text = request.form.get("precio", "").strip()
```

```

        if not name:
            flash("El nombre es obligatorio.", "error")
            return render_template("productos.html", products=products), 400

        try:
            price = float(price_text)
        except ValueError:
            price = 0

        if price <= 0:
            flash("El precio debe ser mayor que cero.", "error")
            return render_template("productos.html", products=products), 400

        products.append({"nombre": name, "precio": f"{price:.2f}"})
        session["products"] = products
        flash("Producto creado.", "success")
        return redirect(url_for("products_view"))

    return render_template("productos.html", products=products)

return app

app = create_app()

```

Plantilla Jinja: templates/productos.html

```

{% extends "base.html" %}

{% block title %}Productos | Inventario{% endblock %}

{% block content %}
<h1>Productos</h1>

<form method="post" aria-label="Crear producto">
  <label for="nombre">Nombre</label>
  <input id="nombre" name="nombre" required>

  <label for="precio">Precio</label>
  <input id="precio" name="precio" type="number" step="0.01" required>

  <button type="submit">Guardar producto</button>
</form>

<section aria-label="Lista de productos" data-testid="lista-productos">
  {% if products %}
  <ul>
    {% for product in products %}

```

```

        <li data-testid="producto-row">
            <span>{{ product.nombre }}</span>
            <span>{{ product.precio }}</span>
        </li>
    {% endfor %}
</ul>
{% else %}
    <p>No hay productos.</p>
{% endif %}
</section>
{% endblock %}

```

Configuración: playwright.flask.config.js

```

import { defineConfig } from "@playwright/test";

export default defineConfig({
  testDir: "./tests/e2e",
  testMatch: "**/flask_jinja.spec.js",
  timeout: 10_000,
  use: {
    baseURL: "http://127.0.0.1:5017",
    screenshot: "only-on-failure",
    trace: "retain-on-failure",
    video: "retain-on-failure"
  },
  webServer: {
    command: "python -m flask --app flask_jinja_demo/app.py run --host 127.0.0.1 -",
    port: 5017,
    url: "http://127.0.0.1:5017",
    reuseExistingServer: !process.env.CI,
    timeout: 120_000
  }
});

```

Spec E2E: tests/e2e/flask_jinja.spec.js

```

import { test, expect } from "@playwright/test";

test.describe("Inventario Flask y Jinja", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("/productos");
  });

  test("renderiza la lista inicial desde una plantilla Jinja", async ({ page }) => {
    await expect(page).toHaveTitle(/Productos \| Inventario/);
    await expect(page.getByRole("heading", { name: "Productos" })).toBeVisible();
    await expect(page.getByTestId("lista-productos")).toContainText("No hay productos");
  });
});

```

```

test("crea un producto y sigue el redirect de Flask", async ({ page }) => {
  await page.getByLabel("Nombre").fill("Teclado");
  await page.getByLabel("Precio").fill("45");
  await page.getByRole("button", { name: "Guardar producto" }).click();

  await expect(page).toHaveURL(/\/productos$/);
  await expect(page.getByRole("status")).toContainText("Producto creado");
  await expect(page.getByTestId("producto-row")).toContainText("Teclado");
});

test("muestra el error del servidor para un precio invalido", async ({ page })
=> {
  await page.getByLabel("Nombre").fill("Producto de prueba");
  await page.getByLabel("Precio").fill("-5");
  await page.getByRole("button", { name: "Guardar producto" }).click();

  await expect(page.getByRole("alert")).toContainText(/mayor que cero/i);
  await expect(page.getByTestId("lista-productos")).toContainText("No hay
productos");
});
});

```

Regla de separación

Prueba con `client.get()` y `client.post()` las reglas del servidor, validaciones y códigos HTTP. Prueba con Playwright los caminos críticos que una persona ejecuta en el navegador. El mismo requisito puede tener ambas pruebas, pero cada una aporta una evidencia diferente.

Componente	Orientación
Ambiente requerido	Aula de informática con computador, navegador y acceso a internet.
Estrategia o técnica didáctica activa	Taller guiado, TDD, revisión por pares y simulación.
Materiales de formación	Ecosistema de código de ejemplo y guía web 5 con simuladores.
Material de apoyo	Ejemplos resueltos y lista de validación del paso a paso.
Evidencia de aprendizaje	Suite automatizada con cobertura $\geq 80\%$ y bitácora del ciclo TDD.
Instrumento de evaluación	Rúbrica de pruebas automatizadas.
Duración	20 horas

4.4 Transferencia: pipeline QA y auditoría del proyecto

Propósito

Integrar la calidad en el ciclo de vida del proyecto: compuertas en GitHub Actions, auditoría multidimensional del código (incluido el generado con IA) y observabilidad post-despliegue.

Reto

Convertir la suite de la actividad 4.3, de un conjunto de pruebas en el computador, a una compuerta de calidad que impide fusionar código defectuoso en la rama principal.

Cobertura de código y compuertas de calidad

Umbral obligatorio ADSO: cobertura mayor o igual al 80 % en la lógica de negocio. Comandos: `pytest --cov=app --cov-fail-under=80`, `npm run test:coverage` y `mvn test jacoco:report`.

CI/CD, Testing con GitHub Actions y versionamiento en Git

- Relación entre Testing y GitHub: GitHub es el orquestador y árbitro imparcial de la calidad. A través de GitHub Actions, las pruebas se ejecutan en ejecutores limpios e independientes por cada push y pull request. Las reglas de protección de rama impiden fusionar cambios a main si las pruebas fallan.
- ¿Se deben subir las pruebas a Git? Sí. Las pruebas son ciudadanos de primera clase (First-Class Citizens). Sin ellas en el repositorio, los ejecutores de CI/CD no tienen nada que ejecutar y no hay reproducibilidad ni trazabilidad con `git bisect`. NUNCA se suben: reportes efímeros (`htmlcov/`), cachés (`.pytest_cache/`) ni secretos (`.env`).
- Seguridad y atacantes: el mito de la 'seguridad por oscuridad' es falso. Ocultar las pruebas no protege la aplicación; los atacantes usan herramientas automáticas (Burp Suite, OWASP ZAP) para encontrar fallas. El riesgo real es subir credenciales quemadas o datos reales en fixtures: usa datos sintéticos con Faker y GitHub Secrets.
- Repositorios Públicos vs Privados: para aprendices, los repositorios públicos son la mejor vitrina de calidad y cobertura. Para empresas y software propietario con acuerdo de confidencialidad, se usan repositorios privados. Regla de oro: escribe y prueba todo proyecto como si fuera a ser público mañana.

Paso a paso para el pipeline CI/CD:

Paso 1: Workflow YAML. Definir `.github/workflows/ci.yml` con eventos de push y pull_request hacia main.

```
> git push origin main
```

Regla de oro: Configura el caché de dependencias para acelerar el pipeline.

Paso 2: Compuerta de calidad. Encadenar linter, pruebas unitarias, integración y validación del umbral del 80 %.

```
> pytest --cov=app --cov-fail-under=80
```

Regla de oro: Si una prueba falla, el despliegue al servidor VPS se cancela de inmediato.

Paso 3: Prueba posterior al despliegue. Ejecutar curl a /health después del despliegue con reversión automática en caso de caída.

```
> curl -f http://localhost:8000/health
```

Regla de oro: Asegura cero tiempo de inactividad y recuperación automática.

```
name: CI - Quality Gate multi-lenguaje

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ci-{{ github.workflow }}-{{ github.ref }}
  cancel-in-progress: true

jobs:
  python-tests:
    name: Python ${{ matrix.python-version }} · PyTest + Behave
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12"]
    defaults:
      run:
        working-directory: .
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip
          cache-dependency-path: requirements.txt
      - name: Instalar dependencias Python
        run: python -m pip install -r requirements.txt
      - name: Ejecutar PyTest con cobertura
        run: python -m pytest tests --cov=services --cov=routers --cov=schemas --
cov-fail-under=80 --cov-report=xml --cov-report=term-missing
      - name: Ejecutar escenarios BDD
        run: python -m behave -q
      - name: Guardar cobertura Python
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: python-coverage-{{ matrix.python-version }}
          path: coverage.xml
```

```
if-no-files-found: ignore
```

javascript-tests:

```
name: JavaScript · Vitest + React Testing Library
```

```
runs-on: ubuntu-latest
```

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
 - with:
 - node-version: "20"
 - cache: npm
 - cache-dependency-path: package-lock.json
- name: Instalar dependencias JavaScript
 - run: npm ci
- name: Ejecutar pruebas y compuerta de cobertura
 - run: npm run test:coverage
- name: Guardar cobertura JavaScript
 - if: always()
 - uses: actions/upload-artifact@v4
 - with:
 - name: javascript-coverage
 - path: coverage/
 - if-no-files-found: ignore

java-tests:

```
name: Java · JUnit 5 + JaCoCo
```

```
runs-on: ubuntu-latest
```

steps:

- uses: actions/checkout@v4
- uses: actions/setup-java@v4
 - with:
 - distribution: temurin
 - java-version: "21"
 - cache: maven
- name: Ejecutar pruebas Java y generar cobertura
 - run: mvn --batch-mode test jacoco:report
- name: Guardar reporte JaCoCo
 - if: always()
 - uses: actions/upload-artifact@v4
 - with:
 - name: java-jacoco
 - path: target/site/jacoco/
 - if-no-files-found: ignore

e2e-tests:

```
name: E2E · Playwright + Flask/Jinja
```

```
runs-on: ubuntu-latest
```

```
needs: [python-tests, javascript-tests]
```

steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
 - with:

```

    node-version: "20"
    cache: npm
    cache-dependency-path: package-lock.json
  - uses: actions/setup-python@v5
    with:
      python-version: "3.12"
      cache: pip
      cache-dependency-path: requirements.txt
  - name: Instalar dependencias
    run: |
      python -m pip install -r requirements.txt
      npm ci
  - name: Instalar Chromium para Playwright
    run: npx playwright install --with-deps chromium
  - name: Ejecutar pruebas E2E
    run: |
      export FLASK_SECRET_KEY="$(python -c 'import secrets; print(secrets.

```

4.4.1 Tests para Git: enganches de pre-commit y detección de secretos

Los enganches de pre-commit ejecutan validaciones en el computador local antes de permitir el commit. Con Gitleaks bloquean la subida accidental de llaves API y con PyTest ejecutan pruebas unitarias rápidas (menos de 2 segundos).

```

# =====
# .pre-commit-config.yaml – Quality Gate Local en Git
# Guía de Testing & QA Universal (SENA ADSO)
# =====
# Propósito: Ejecutar pruebas estáticas, formateo, escaneo de secretos y tests
# unitarios rápidos en la máquina del desarrollador ANTES de permitir el commit.
#
# Instalación:
#   pip install pre-commit gitleaks
#   pre-commit install
# =====

repos:
  # 1. Higiene básica del repositorio Git
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
        name: 🧹 Limpiar espacios en blanco al final de línea
      - id: end-of-file-fixer
        name: 📄 Asegurar salto de línea al final del archivo
      - id: check-yaml
        name: ⚙️ Validar sintaxis de archivos YAML (.github/workflows)
      - id: check-json
        name: 🔍 Validar sintaxis de archivos JSON
      - id: check-added-large-files
        name: 📦 Prevenir subida de binarios grandes (>500KB)
        args: ['--maxkb=500']

```

```

- id: check-merge-conflict
  name: 🚫 Bloquear marcadores de conflicto de merge olvidados

# 2. Seguridad en Git: Detección estricta de secretos y credenciales
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.4
  hooks:
    - id: gitleaks
      name: 🛡️ Gitleaks: Prevenir subida de contraseñas, tokens y llaves
      privadas

# 3. Linter y verificación estática rápida (Python)
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.4.4
  hooks:
    - id: ruff
      name: ⚡ Ruff: Linter ultrarrápido de código Python
      args: [--fix]

# 4. Pruebas Unitarias Rápidas Locales (Smoke Gate antes del Commit)
- repo: local
  hooks:
    - id: pytest-unit-fast
      name: 🧪 PyTest: Ejecución de pruebas unitarias críticas (< 2s)
      entry: pytest tests/unit -q --tb=line
      language: system
      types: [python]
      pass_filenames: false

```

4.4.3 Auditoría del código generado con IA (qa_auditor)

Sistema en recursos/auditoria-seguridad/ que mide 9 dimensiones de calidad y detecta anomalías en código generado con IA: pruebas vacías, imports alucinados y vulnerabilidades OWASP. Comando: `python -m qa_auditor --target ../codigo-ejemplo --url http://localhost:8000 -min-score 80`

4.4.4 Observabilidad post-despliegue y triaje con IA

Paso a paso para observabilidad y diagnóstico en VPS:

Paso 1: Registros JSON y correlación. Configurar JSONFormatter en Flask inyectando X-Request-ID y endpoints /healthz, /readyz y /metrics.

```
> python recursos/observabilidad/app_flask_ejemplo.py
```

Regla de oro: Los registros JSON evitan errores de análisis y facilitan el procesamiento por agentes de IA.

Paso 2: Alertas en Coolify. Activar en Coolify notificaciones de fallo de despliegue y límite de reinicios vía Discord o Webhook.

```
> curl -X POST http://localhost:9050/webhook/coolify
```

Regla de oro: Los límites de reinicios detectan ciclos de caída antes de saturar el servidor VPS.

Paso 3: Visor Dozzle. Desplegar Dozzle montando /var/run/docker.sock para inspeccionar registros en vivo desde la web.

```
> docker compose -f recursos/observabilidad/docker-compose.observability.yml up -d
```

Regla de oro: Dozzle consume menos de 15 MB de RAM y permite búsquedas con expresiones regulares.

Paso 4: Agente SRE con IA. Ejecutar ai_log_watcher para deduplicar incidentes con SHA-256 y generar diagnóstico RCA.

```
> python recursos/observabilidad/ai_log_watcher.py --test
```

Regla de oro: El mensaje estructurado devuelve severidad, causa raíz y el parche de código exacto.

Componente	Orientación
Ambiente requerido	Aula de informática con repositorio remoto y plataforma virtual.
Estrategia o técnica didáctica activa	Aprendizaje por proyectos, trabajo colaborativo y sustentación.
Materiales de formación	Repositorio GitHub, GitHub Actions, qa_auditor y recursos de observabilidad.
Material de apoyo	Guía web 5, ejemplo resuelto y acompañamiento del instructor.
Evidencia de aprendizaje	Registro integral de evidencias y sustentación del pipeline.
Instrumento de evaluación	Rúbrica analítica de producto y lista de chequeo de desempeño.
Duración	10 horas

4.5 Ruta AA12 a AA16 para documentar las pruebas

Esta ruta convierte cada prueba en un artefacto verificable. El aprendiz trabaja sobre una versión identificada del proyecto, conserva los resultados reales y relaciona requisito, riesgo, caso, ejecución, defecto y decisión. Las plantillas son una adaptación didáctica de la documentación de pruebas; el instructor confirma el formato y el instrumento institucional aplicable.

Código	Actividad	Producto mínimo	Criterio de cierre
AA12	Realizar plan de pruebas	Plan, alcance, riesgos y estrategia	Cada requisito en alcance tiene criterio y riesgo.
AA13	Definir casos de prueba	Matriz de casos y pruebas automatizadas	Los casos cubren éxito, error, límites y permisos.
AA14	Definir ambiente de prueba	Ficha del ambiente y datos sintéticos	Otra persona puede reproducir la preparación.
AA15	Realizar pruebas	Registro de ejecución, reportes y defectos	Cada resultado tiene comando, evidencia y estado.
AA16	Documentar las pruebas	Informe final y Portafolio de Evidencias	La entrega es trazable, legible y verificable.

AA12 Realizar plan de pruebas

Paso 1: Identificar el objeto de prueba. Registra el nombre del sistema, versión o compromiso de Git, alcance, responsables, fecha y componentes que serán evaluados.

Paso 2: Reunir requisitos y criterios. Copia cada requisito funcional y no funcional con un identificador estable y escribe qué condición observable significa que está cumplido.

Paso 3: Analizar riesgos. Valora probabilidad e impacto, prioriza los riesgos críticos y define la prueba o control que reducirá cada uno.

Paso 4: Definir la estrategia. Selecciona niveles, tipos, técnicas, herramientas, roles, datos, dependencias, restricciones y orden de ejecución. Declara qué queda fuera del alcance.

Paso 5: Fijar criterios de entrada y salida. Especifica qué debe estar listo para comenzar y qué condiciones permiten cerrar: resultados, defectos críticos, cobertura acordada y riesgos residuales. No uses un porcentaje como garantía universal.

Paso 6: Construir la trazabilidad. Relaciona requisito, riesgo, caso de prueba, archivo de prueba, ejecución, defecto y evidencia. Revisa que no haya requisitos sin prueba ni pruebas sin propósito.

Paso 7: Revisar y versionar. Solicita revisión del instructor o del equipo, corrige observaciones y guarda el documento como docs/pruebas/01-plan-pruebas-29119-3.md.

AA13 Definir casos de prueba

Paso 1: Derivar casos de cada requisito. Incluye el flujo exitoso, entradas inválidas, límites, permisos, estados, duplicación o conflicto cuando sean riesgos del proyecto.

Paso 2: Asignar un identificador. Usa una nomenclatura estable, por ejemplo CP-001, y clasifica prioridad, nivel de prueba, técnica y requisito cubierto.

Paso 3: Describir el caso completo. Diligencia precondiciones, datos sintéticos, pasos, resultado esperado, oráculo, limpieza y evidencia que debe conservarse.

Paso 4: Elegir la forma de ejecución. Indica si será manual, unitaria, integración, contrato, E2E o no funcional; ubica el archivo automatizado y el comando que lo ejecuta.

Paso 5: Revisar la matriz. Elimina duplicados, identifica casos bloqueados y confirma que las aserciones comprueban resultados y efectos, no solo que el código se ejecutó. Guarda docs/pruebas/02-casos-prueba.md.

AA14 Definir ambiente de prueba

Paso 1: Especificar requisitos técnicos. Registra sistema operativo, versiones de lenguaje y dependencias, navegador, base de datos, servicios, red y recursos necesarios.

Paso 2: Preparar un ambiente aislado. Usa un entorno virtual, contenedor o configuración reproducible. Nunca utilices datos reales ni guardes secretos en el repositorio.

Paso 3: Registrar configuración segura. Documenta nombres de variables, archivos de configuración y valores de ejemplo; conserva las credenciales fuera del código y del informe.

Paso 4: Crear y reiniciar datos de prueba. Define datos sintéticos, usuarios por rol, estados iniciales, semillas y procedimiento de limpieza o reversión.

Paso 5: Ejecutar una comprobación de humo. Verifica instalación, salud del servicio, acceso a dependencias y un caso básico antes de interpretar fallos funcionales.

Paso 6: Guardar la ficha del ambiente. Conserva versiones, fecha, responsable, limitaciones y resultado de la comprobación en docs/pruebas/03-ambiente-prueba.md.

AA15 Realizar pruebas

Paso 1: Congelar la versión evaluada. Anota rama, compromiso de Git, fecha y cambios pendientes. Si la versión cambia, abre una nueva ejecución o explica la diferencia.

Paso 2: Ejecutar por niveles. Corre primero las pruebas unitarias, luego integración o contrato y finalmente los recorridos E2E y no funcionales que estén en alcance.

Paso 3: Registrar el resultado real. Para cada caso escribe aprobado, fallido o bloqueado, comando, fecha, duración, salida relevante, resultado esperado y resultado observado.

Paso 4: Separar ambiente y producto. Si falla la preparación, corrige el ambiente y repite. Si el entorno está correcto y el observado contradice el oráculo, registra un defecto reproducible.

Paso 5: Corregir y repetir. Conserva la evidencia del fallo, la corrección, la prueba de confirmación y la regresión de los casos relacionados. No borres una ejecución fallida.

Paso 6: Conservar resultados. Guarda el registro en docs/pruebas/04-registro-ejecuciones.md y los reportes, capturas, trazas o videos seleccionados bajo test-results/.

AA16 Documentar las pruebas

Paso 1: Consolidar las métricas. Resume casos planificados, ejecutados, aprobados, fallidos y bloqueados; cobertura interpretada, defectos por severidad y resultados por nivel.

Paso 2: Documentar hallazgos y ausencia de hallazgos. Cada defecto debe tener pasos, esperado, observado, severidad, prioridad, estado, responsable y evidencia. Si no hay defectos, declara el alcance probado y sus límites.

Paso 3: Cerrar contra los criterios. Concluye si se cumplen los criterios de salida, qué riesgos permanecen, qué no se probó y qué recomendación corresponde al responsable de la entrega.

Paso 4: Armar el Portafolio de Evidencias. Relaciona cada artefacto con su criterio, instrumento, enlace relativo y resultado. Completa el registro local y la bitácora docs/ai-log.md si utilizaste una herramienta de IA.

Paso 5: Revisar la entrega. Comprueba enlaces, nombres, versión, ortografía, ausencia de secretos, reproducibilidad de comandos y correspondencia entre el informe y los archivos reales.

Paso 6: Empaquetar y entregar. Genera docs/pruebas/06-informe-final-pruebas.md, incluye código de pruebas y evidencias necesarias, excluye credenciales y temporales, y entrega el paquete por el canal definido por el instructor.

Estructura mínima del Portafolio de Evidencias

```
docs/pruebas/  
├── 01-plan-pruebas-29119-3.md  
├── 02-casos-prueba.md  
├── 03-ambiente-prueba.md  
├── 04-registro-ejecuciones.md  
├── 05-registro-defectos.md  
├── 06-informe-final-pruebas.md  
└── 07-matriz-trazabilidad.csv  
tests/                # código automatizado por nivel  
test-results/         # reportes y evidencias reproducibles  
docs/ai-log.md        # apoyo de IA, verificación y decisión humana
```

Lista de verificación antes de entregar

- El plan declara alcance, riesgos, estrategia, criterios de entrada y salida, ambiente y límites.

- Cada requisito en alcance tiene al menos un caso y cada caso tiene un resultado esperado comprobable.
- El ambiente se puede preparar con datos sintéticos sin revelar secretos ni datos personales.
- La ejecución conserva comandos, versión, resultado, reportes y defectos; los fallos no fueron ocultados.
- El informe identifica riesgos residuales, elementos no probados y la relación entre evidencia y criterio.
- La entrega contiene archivos editables, enlaces relativos funcionales, nombres estables y el formato solicitado por el instructor.

5. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN

La evaluación es formativa: la retroalimentación debe señalar evidencia, criterio afectado y acción de mejora. El juicio final considera conocimiento, desempeño y producto; completar una actividad interactiva no reemplaza la verificación del entregable.

Fase del proyecto formativo	Actividad de aprendizaje	Evidencias de aprendizaje	Criterios de evaluación	Técnicas e instrumentos
Evaluación	4.2 Contextualización	Conocimiento: cuestionario y explicación de la pirámide de pruebas.	Diferencia pruebas unitarias, de integración y end-to-end, y justifica sus proporciones.	Cuestionario y preguntas de comprensión.
Evaluación	4.3 Apropiación	Desempeño: construcción y ejecución de la suite automatizada con TDD.	Aplica el ciclo rojo, verde, refactor y mide la cobertura con $\geq 80\%$.	Observación directa y rúbrica de pruebas automatizadas.
Evaluación	4.4 Transferencia	Producto: registro integral de evidencias con plan de pruebas y pipeline.	El informe es trazable, verificable y útil para la entrega del proyecto.	Rúbrica analítica de producto y sustentación.

5.1 Criterios de evaluación aplicados

1. Identifica los tipos de prueba de acuerdo con la pirámide de testing y sus criterios de aceptación.
2. Ejecuta pruebas automatizadas unitarias, de integración y end-to-end siguiendo una metodología establecida.
3. Aplica el ciclo TDD y la especificación BDD en funcionalidades del proyecto formativo.
4. Integra la calidad en el pipeline CI/CD y en las reglas de protección de la rama principal.
5. Documenta el plan de pruebas (IEEE 829) y sustenta los hallazgos con evidencia real.
6. Detecta ausencia de niveles de cobertura y usa auditorías estáticas para identificar vulnerabilidades.

5.2 Rúbrica analítica de aceptación

Para cada criterio el instructor registra: Cumple, Cumple parcialmente o Aún no cumple, acompañado de una observación verificable y una acción de mejora.

Criterio	Indicador observable	Carácter
Plan de pruebas (IEEE 829)	Alcance, estrategia y matriz de casos definidos antes de codificar la suite.	Esencial
Cobertura $\geq 80\%$	La lógica de negocio alcanza el umbral y el reporte lo demuestra.	Esencial
TDD rojo, verde, refactor	Las pruebas se escribieron antes que la implementación	Esencial

	y hay bitácora del ciclo.	
Pipeline CI/CD	La compuerta de bloqueo funciona en un repositorio real y se evidencia con una captura.	Esencial
Pruebas end-to-end	Los caminos dorados se ejecutan en navegador real y quedan trazas de los fallos.	Esencial
Calidad y seguridad	La auditoría detecta anomalías del código y no hay secretos en el repositorio.	Esencial
Comunicación	El registro integral es claro, legible y puede explicarse con vocabulario técnico.	Complementario
Uso ético de la IA	Registra el apoyo de herramientas de IA y verifica con evidencia cada resultado.	Complementario

5.3 Reglas de entrega

- Usar nombres de archivo estables: G05a_ApellidoNombre_Artefacto_vNN.
- Entregar archivos editables y una versión de consulta cuando aplique.
- Citar fuentes, diferenciar el contenido propio y registrar el apoyo de herramientas de IA.
- No incluir datos personales, credenciales ni información sensible sin autorización.
- Aplicar la retroalimentación y conservar el historial de versiones.

6. AMBIENTES DE APRENDIZAJE

La guía se desarrolla en ambientes presenciales y virtuales con acceso al ecosistema de código y a la plataforma interactiva.

Ambiente	Recursos físicos	Recursos tecnológicos	Recursos digitales
Aula de informática / laboratorio de sistemas	Computadores, tablero, video proyector.	Python 3.11+, Node.js 20+, JDK 17, venv y gestores de paquetes.	Plataforma interactiva de la guía (módulos y simuladores), repositorio GitHub y GitHub Actions.
Plataforma virtual de aprendizaje	Puesto de trabajo con conexión a internet.	Navegador web actualizado.	Guía web, foros, entrega de evidencias y rúbricas.
Entorno de trabajo autónomo	Computador propio.	Python, Node.js, JDK y Docker (opcional).	Recursos de la guía, videos y actividades de refuerzo.

7. GLOSARIO DE TÉRMINOS

Término	Definición
Bug	Error en el comportamiento del software detectado en ejecución.
Cobertura de código	Porcentaje de líneas o ramas ejecutadas por la suite de pruebas.
Calidad de software	Grado de cumplimiento de los requisitos funcionales y de atributos (ISO/IEC 25010).
CI/CD	Integración y entrega continua: automatización del build, las pruebas y el despliegue.
Prueba unitaria	Prueba aislada de la lógica pura sin dependencias externas, rápida y determinista.
Prueba de integración	Prueba que valida la interacción de módulos, APIs y base de datos.
Prueba end-to-end (E2E)	Prueba del flujo completo del usuario en navegador o red real.
Pirámide de pruebas	Modelo que prioriza base amplia de pruebas unitarias sobre integración y E2E.
Plan de pruebas IEEE 829	Documento que define alcance, estrategia y matriz de casos de prueba.
Quality Gate	Compuerta de calidad que bloquea la fusión o el despliegue cuando la evidencia falla.
TDD	Desarrollo guiado por pruebas: se escribe primero la prueba que falla antes de la implementación.
BDD	Comportamiento guiado por historias: especificación con formato Dado, Cuando, Entonces.
Protocolo V.E.R.A.	Verificar en documentos, Ejecutar con TDD, Revisar calidad y seguridad, Atribuir en bitácora.
Dato sintético	Dato artificial generado con Faker para no exponer información real en las pruebas.
Trazabilidad	Capacidad de rastrear un defecto desde la evidencia hasta el código que lo causó.

8. REFERENTES BIBLIOGRÁFICOS

Referentes curriculares, técnicos y de apoyo consultados para la elaboración de la guía:

- Servicio Nacional de Aprendizaje - SENA. (2021). Programa de formación titulada: Análisis y Desarrollo de Software, código 228118.
- Servicio Nacional de Aprendizaje - SENA. (s. f.). Formato Guía de Aprendizaje GFPI-F-135.
- Institute of Electrical and Electronics Engineers. (2008). IEEE 829: Standard for Software and System Test Documentation.
- International Organization for Standardization. (2011). ISO/IEC 25010: Systems and software Quality Requirements and Evaluation (SQuaRE).
- Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley.
- Cohn, M. (2009). Succeeding with Agile. Addison-Wesley.
- North, D. (2006). Introducing BDD. Better Software Magazine.
- Playwright. (2024). Documentación oficial. <https://playwright.dev/docs>
- Playwright. (s. f.). Documentación de localizadores, fixtures y servidor web. <https://playwright.dev/docs/locators>
- Pallets Projects. (s. f.). Flask: Testing Flask Applications. <https://flask.palletsprojects.com/en/stable/testing/>
- Pallets Projects. (s. f.). Flask: Templates. <https://flask.palletsprojects.com/en/stable/templates/>
- PyTest. (2024). Documentación oficial. <https://docs.pytest.org/>
- Jest. (2024). Documentación oficial. <https://jestjs.io/docs>
- JUnit 5. (2024). Documentación oficial. <https://junit.org/junit5/docs/>
- GitHub Actions. (2024). Documentación oficial. <https://docs.github.com/actions>
- OWASP. (2024). Web Security Testing Guide. <https://owasp.org/www-project-web-security-testing-guide/>
- Microsoft. (2024). Playwright Best Practices.

9. CONTROL DEL DOCUMENTO

Rol	Nombre	Cargo	Dependencia	Fecha
Autor(es)	Instructor — SENA ADSO	Instructor	Centro de formación / coordinación académica	10 de septiembre de 2026
Revisó	Por diligenciar	Coordinador académico	Centro de formación	Por diligenciar
Aprobó	Por diligenciar	Responsable del programa ADSO	Centro de formación	Por diligenciar

10. CONTROL DE CAMBIOS

Rol	Nombre	Cargo	Dependencia	Fecha	Razón del cambio
Autor(es)	Instructor — SENA ADSO	Instructor	Centro de formación	10 de septiembre de 2026	Ajuste de la adaptación didáctica (versión 3.0.0): enseñanza por proyectos, secuencia reflexión-apropiación, evidencias y evaluación, ambientes, glosario y ortografía es-CO.